实验三寄存器堆功能改进实验报告

2026 年 4 月 21 日

1 实验目的

1. 理解 32 位通用寄存器堆的硬件结构与读写工作原理。
2. 将写使能信号从 1 位扩展为 2 位，实现高低 16 位独立写入控制。
3. 新增专用寄存器，实时计算并存储 1 号与 2 号寄存器数据之和。
4. 完成 FPGA 板级验证，验证改进后寄存器堆功能正确性。

2 实验原理与改进方案

2.1 原有寄存器堆结构

传统寄存器堆包含 32 个 32 位寄存器，两路异步读、一路同步写，使用 1 位写使能控制整个 32 位数据写入。

2.2 本次改进内容

1. **写使能扩展为 2 位**
   - `wen[0]`：控制写数据低 16 位 `wdata[15:0]`
   - `wen[1]`：控制写数据高 16 位 `wdata[31:16]`

   可实现只写高 16 位、只写低 16 位或完整 32 位写入。
2. **新增专用求和寄存器**增加寄存器 `sum_reg`，在时钟上升沿自动计算

$$\text{sum\_reg} = \text{rf}[1] + \text{rf}[2]$$

3. **调试端口扩展**将测试地址 `test_addr = 31` 映射为 `sum_reg`，可直接观察求和结果。

2.3 改进后寄存器堆原理图说明

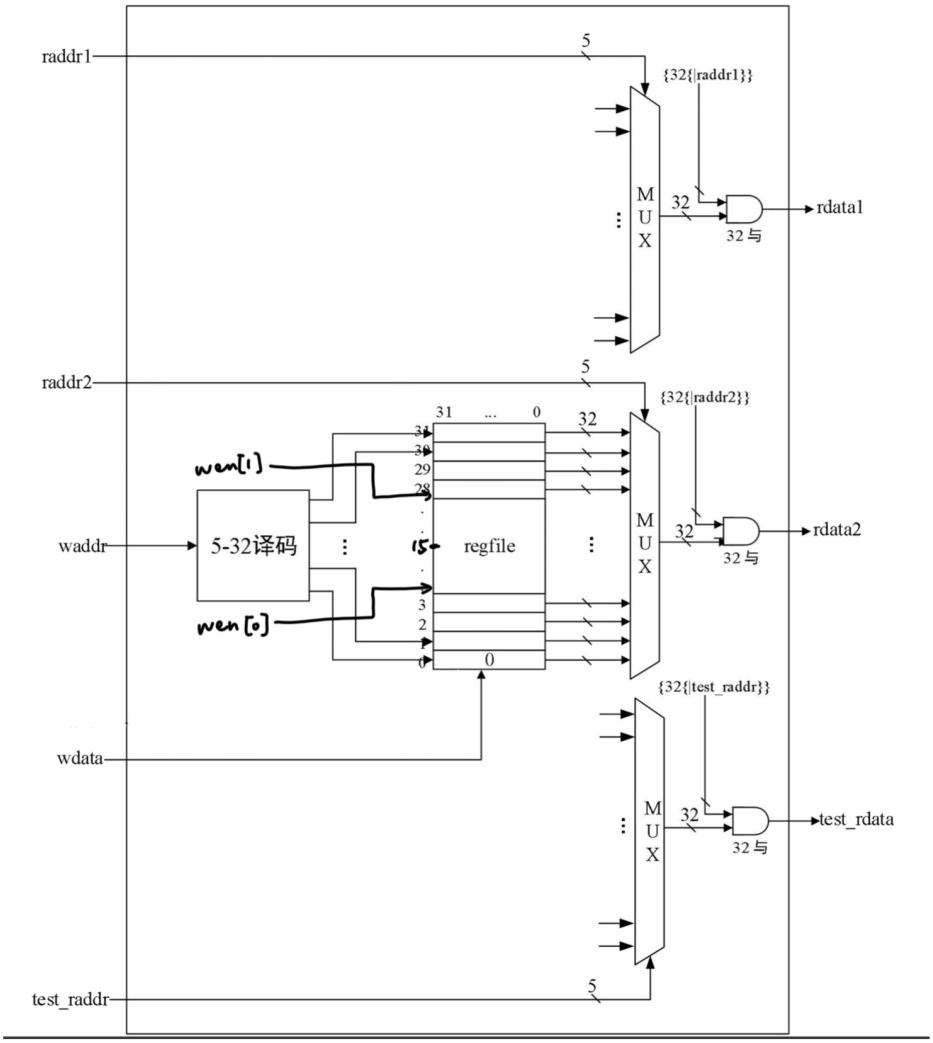

图 1: 改进后寄存器堆硬件原理图

原理图结构说明：

1. 写控制逻辑：2 位写使能分别控制高低 16 位写入使能。
2. 加法模块：时钟同步加法器，实时计算 rf[1]+rf[2]。
3. 所有写操作与求和操作均在时钟上升沿同步完成。

3 改进后 Verilog 代码实现

3.1 寄存器堆核心模块 regfile.v

```
`timescale 1ns / 1ps
module regfile(
    input              clk,
    input       [1:0]  wen, // 2位写使能
    input       [4 :0] raddr1,
    input       [4 :0] raddr2,
    input       [4 :0] waddr,
    input       [31:0] wdata,
    output reg  [31:0] rdata1,
    output reg  [31:0] rdata2,
    input       [4 :0] test_addr,
    output reg  [31:0] test_data
);
    reg [31:0] rf[31:0];
    reg [31:0] sum_reg;  // 新增求和寄存器

    // 分高低16位写入
    always @(posedge clk)
    begin
        if(waddr != 5'd0) begin
            if(wen[0]) rf[waddr][15:0]  <= wdata[15:0];
            if(wen[1]) rf[waddr][31:16] <= wdata[31:16];
        end
        sum_reg <= rf[1] + rf[2];
    end

    always @(*)
    begin
        case (raddr1)
            5'd1 : rdata1 <= rf[1 ];
            5'd2 : rdata1 <= rf[2 ];
            5'd3 : rdata1 <= rf[3 ];
            5'd4 : rdata1 <= rf[4 ];
            5'd5 : rdata1 <= rf[5 ];
            5'd6 : rdata1 <= rf[6 ];
            5'd7 : rdata1 <= rf[7 ];
            5'd8 : rdata1 <= rf[8 ];
            5'd9 : rdata1 <= rf[9 ];
            5'd10: rdata1 <= rf[10];
```

```
            5'd11: rdata1 <= rf[11];
            5'd12: rdata1 <= rf[12];
            5'd13: rdata1 <= rf[13];
            5'd14: rdata1 <= rf[14];
            5'd15: rdata1 <= rf[15];
            5'd16: rdata1 <= rf[16];
            5'd17: rdata1 <= rf[17];
            5'd18: rdata1 <= rf[18];
            5'd19: rdata1 <= rf[19];
            5'd20: rdata1 <= rf[20];
            5'd21: rdata1 <= rf[21];
            5'd22: rdata1 <= rf[22];
            5'd23: rdata1 <= rf[23];
            5'd24: rdata1 <= rf[24];
            5'd25: rdata1 <= rf[25];
            5'd26: rdata1 <= rf[26];
            5'd27: rdata1 <= rf[27];
            5'd28: rdata1 <= rf[28];
            5'd29: rdata1 <= rf[29];
            5'd30: rdata1 <= rf[30];
            5'd31: rdata1 <= rf[31];
            default : rdata1 <= 32'd0;
        endcase
    end

    always @(*)
    begin
        case (raddr2)
            5'd1 : rdata2 <= rf[1 ];
            5'd2 : rdata2 <= rf[2 ];
            5'd3 : rdata2 <= rf[3 ];
            5'd4 : rdata2 <= rf[4 ];
            5'd5 : rdata2 <= rf[5 ];
            5'd6 : rdata2 <= rf[6 ];
            5'd7 : rdata2 <= rf[7 ];
            5'd8 : rdata2 <= rf[8 ];
            5'd9 : rdata2 <= rf[9 ];
            5'd10: rdata2 <= rf[10];
            5'd11: rdata2 <= rf[11];
            5'd12: rdata2 <= rf[12];
```

```
            5'd13: rdata2 <= rf[13];
            5'd14: rdata2 <= rf[14];
            5'd15: rdata2 <= rf[15];
            5'd16: rdata2 <= rf[16];
            5'd17: rdata2 <= rf[17];
            5'd18: rdata2 <= rf[18];
            5'd19: rdata2 <= rf[19];
            5'd20: rdata2 <= rf[20];
            5'd21: rdata2 <= rf[21];
            5'd22: rdata2 <= rf[22];
            5'd23: rdata2 <= rf[23];
            5'd24: rdata2 <= rf[24];
            5'd25: rdata2 <= rf[25];
            5'd26: rdata2 <= rf[26];
            5'd27: rdata2 <= rf[27];
            5'd28: rdata2 <= rf[28];
            5'd29: rdata2 <= rf[29];
            5'd30: rdata2 <= rf[30];
            5'd31: rdata2 <= rf[31];
            default : rdata2 <= 32'd0;
        endcase
    end

    always @(*)
    begin
        case (test_addr)
            5'd1 : test_data <= rf[1 ];
            5'd2 : test_data <= rf[2 ];
            5'd3 : test_data <= rf[3 ];
            5'd4 : test_data <= rf[4 ];
            5'd5 : test_data <= rf[5 ];
            5'd6 : test_data <= rf[6 ];
            5'd7 : test_data <= rf[7 ];
            5'd8 : test_data <= rf[8 ];
            5'd9 : test_data <= rf[9 ];
            5'd10: test_data <= rf[10];
            5'd11: test_data <= rf[11];
            5'd12: test_data <= rf[12];
            5'd13: test_data <= rf[13];
            5'd14: test_data <= rf[14];
```

```
            5'd15: test_data <= rf[15];
            5'd16: test_data <= rf[16];
            5'd17: test_data <= rf[17];
            5'd18: test_data <= rf[18];
            5'd19: test_data <= rf[19];
            5'd20: test_data <= rf[20];
            5'd21: test_data <= rf[21];
            5'd22: test_data <= rf[22];
            5'd23: test_data <= rf[23];
            5'd24: test_data <= rf[24];
            5'd25: test_data <= rf[25];
            5'd26: test_data <= rf[26];
            5'd27: test_data <= rf[27];
            5'd28: test_data <= rf[28];
            5'd29: test_data <= rf[29];
            5'd30: test_data <= rf[30];
            5'd31: test_data <= sum_reg; // 设置专用寄存器
            default : test_data <= 32'd0;
        endcase
    end
endmodule
```

3.2 显示驱动模块 regfile_display.v

```
module regfile_display(
    input clk,
    input resetn,

    input [1:0] wen, // 2位写使能
    input [1:0] input_sel,

    output led_wen,
    output led_waddr,
    output led_wdata,
    output led_raddr1,
    output led_raddr2,

    output lcd_rst,
    output lcd_cs,
```

```
    output lcd_rs,
    output lcd_wr,
    output lcd_rd,
    inout[15:0] lcd_data_io,
    output lcd_bl_ctr,
    inout ct_int,
    inout ct_sda,
    output ct_scl,
    output ct_rstn
);
    assign led_wen    = |wen; // 控制写使能 LED 指示灯正常工作
    assign led_raddr1 = (input_sel==2'd0);
    assign led_raddr2 = (input_sel==2'd1);
    assign led_waddr  = (input_sel==2'd2);
    assign led_wdata  = (input_sel==2'd3);

    wire [31:0] test_data;
    wire [4 :0] test_addr;
    reg  [4 :0] raddr1;
    reg  [4 :0] raddr2;
    reg  [4 :0] waddr;
    reg  [31:0] wdata;
    wire [31:0] rdata1;
    wire [31:0] rdata2;

    regfile rf_module(
        .clk      (clk     ),
        .wen      (wen     ),
        .raddr1   (raddr1  ),
        .raddr2   (raddr2  ),
        .waddr    (waddr   ),
        .wdata    (wdata   ),
        .rdata1   (rdata1  ),
        .rdata2   (rdata2  ),
        .test_addr(test_addr),
        .test_data(test_data)
    );

    reg         display_valid;
    reg  [39:0] display_name;
```

```
56      reg  [31:0] display_value;
57      wire [5 :0] display_number;
58      wire        input_valid;
59      wire [31:0] input_value;
60
61      lcd_module lcd_module(
62          .clk            (clk           ),
63          .resetn         (resetn        ),
64          .display_valid  (display_valid ),
65          .display_name   (display_name  ),
66          .display_value  (display_value ),
67          .display_number (display_number),
68          .input_valid    (input_valid   ),
69          .input_value    (input_value   ),
70          .lcd_rst        (lcd_rst       ),
71          .lcd_cs         (lcd_cs        ),
72          .lcd_rs         (lcd_rs        ),
73          .lcd_wr         (lcd_wr        ),
74          .lcd_rd         (lcd_rd        ),
75          .lcd_data_io    (lcd_data_io   ),
76          .lcd_bl_ctr     (lcd_bl_ctr    ),
77          .ct_int         (ct_int        ),
78          .ct_sda         (ct_sda        ),
79          .ct_scl         (ct_scl        ),
80          .ct_rstn        (ct_rstn       )
81      );
82
83      assign test_addr = display_number - 5'd7;
84
85      always @(posedge clk) begin
86          if (!resetn) raddr1 <= 5'd0;
87          else if (input_valid && input_sel == 2'd0) raddr1 <=
                input_value[4:0];
88      end
89
90      always @(posedge clk) begin
91          if (!resetn) raddr2 <= 5'd0;
92          else if (input_valid && input_sel == 2'd1) raddr2 <=
                input_value[4:0];
93      end
```

```
94
95      always @(posedge clk) begin
96          if (!resetn) waddr <= 5'd0;
97          else if (input_valid && input_sel == 2'd2) waddr <=
                input_value[4:0];
98      end
99
100     always @(posedge clk) begin
101         if (!resetn) wdata <= 32'd0;
102         else if (input_valid && input_sel == 2'd3) wdata <=
                input_value;
103     end
104
105     always @(posedge clk) begin
106         if (display_number > 6'd6 && display_number < 6'd39) begin
107             display_valid <= 1'b1;
108             display_name[39:16] <= "REG";
109             display_name[15: 8] <= {4'b0011,3'b000,test_addr[4]};
110             display_name[7 : 0] <= {4'b0011,test_addr[3:0]};
111             display_value <= test_data;
112         end
113         else begin
114             case(display_number)
115                 6'd1: begin display_valid<=1; display_name<="RADD1";
                        display_value<=raddr1; end
116                 6'd2: begin display_valid<=1; display_name<="RDAT1";
                        display_value<=rdata1; end
117                 6'd3: begin display_valid<=1; display_name<="RADD2";
                        display_value<=raddr2; end
118                 6'd4: begin display_valid<=1; display_name<="RDAT2";
                        display_value<=rdata2; end
119                 6'd5: begin display_valid<=1; display_name<="WADDR";
                        display_value<=waddr;  end
120                 6'd6: begin display_valid<=1; display_name<="WDATA";
                        display_value<=wdata;  end
121                 default: begin display_valid<=0; display_name<=0;
                        display_value<=0; end
122             endcase
123         end
124     end
```

```
endmodule
```

3.3 管脚约束文件

```
#时钟信号连接
set_property PACKAGE_PIN AC19 [get_ports clk]

#复位按键，低电平有效
set_property PACKAGE_PIN Y3 [get_ports resetn]

#LED指示灯
set_property PACKAGE_PIN H7 [get_ports led_wen]
set_property PACKAGE_PIN D5 [get_ports led_waddr]
set_property PACKAGE_PIN A3 [get_ports led_wdata]
set_property PACKAGE_PIN A5 [get_ports led_raddr1]
set_property PACKAGE_PIN A4 [get_ports led_raddr2]

#拨码开关 (wen[0] 和 wen[1] 分别分配到具体的硬件引脚)
set_property PACKAGE_PIN AC21 [get_ports wen[0]]
set_property PACKAGE_PIN AD24 [get_ports wen[1]]
set_property PACKAGE_PIN AC22 [get_ports input_sel[0]]
set_property PACKAGE_PIN AC23 [get_ports input_sel[1]]

set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports resetn]
set_property IOSTANDARD LVCMOS33 [get_ports led_wen]
set_property IOSTANDARD LVCMOS33 [get_ports led_raddr1]
set_property IOSTANDARD LVCMOS33 [get_ports led_raddr2]
set_property IOSTANDARD LVCMOS33 [get_ports led_waddr]
set_property IOSTANDARD LVCMOS33 [get_ports led_wdata]
set_property IOSTANDARD LVCMOS33 [get_ports wen]
set_property IOSTANDARD LVCMOS33 [get_ports input_sel[1]]
set_property IOSTANDARD LVCMOS33 [get_ports input_sel[0]]

#LCD接口
set_property PACKAGE_PIN J25 [get_ports lcd_rst]
set_property PACKAGE_PIN H18 [get_ports lcd_cs]
set_property PACKAGE_PIN K16 [get_ports lcd_rs]
set_property PACKAGE_PIN L8 [get_ports lcd_wr]
```

```
set_property PACKAGE_PIN K8 [get_ports lcd_rd]
set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
set_property PACKAGE_PIN L19 [get_ports ct_int]
set_property PACKAGE_PIN J24 [get_ports ct_sda]
set_property PACKAGE_PIN H21 [get_ports ct_scl]
set_property PACKAGE_PIN G24 [get_ports ct_rstn]

set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
```

4 实验步骤

1. 新建 Vivado 工程，添加上述 Verilog 设计文件与约束文件。
2. 完成综合、实现、比特流生成。
3. 将比特流下载至 FPGA 实验箱。
4. 通过拨码开关与按键配置读写地址、数据与写使能。
5. 观察 LCD 显示结果，验证高低 16 位独立写入与求和功能。

5 实验箱验证结果与说明

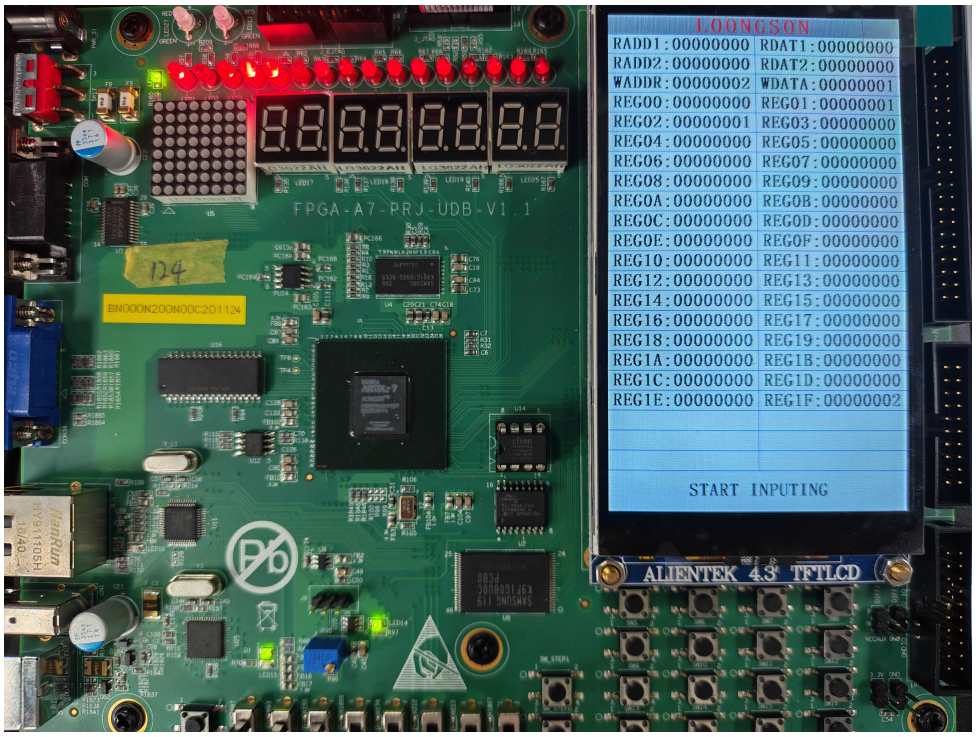

图 2: 添加求和操作

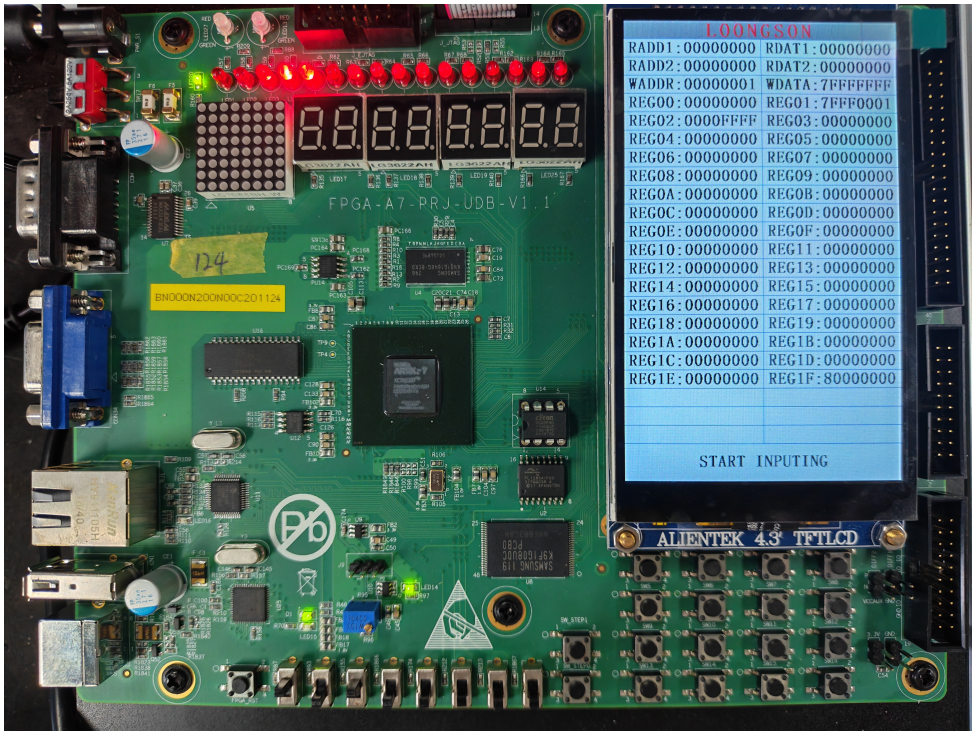

图 3: 改进写操作

5.1 验证内容与数据说明

1. **专用求和寄存器测试**

   - `REG01 + REG02 = REG1F (1 + 1 = 2)`
   - 与理论计算一致，求和功能正确。

2. **高低 16 位独立写入测试 (在实验 1 基础上进行测试)**

   - 写数据：`wdata = 0x7FFFFFFF`
   - 仅使能 `wen[0]=1`，写入低 16 位，REG02 显示为 0x0000FFFF
   - 仅使能 `wen[1]=1`，写入高 16 位，REG01 显示为 0x7FFF0001
   - 经实验可得，独立写入功能正确

6 实验总结

1. 成功将寄存器堆写使能扩展为 2 位，实现高低 16 位独立可控写入。
2. 实现了专用求和寄存器，可实时计算并保存 1 号与 2 号寄存器之和。
3. 完成 FPGA 板级验证，所有功能均正常工作，符合设计要求。
4. 进一步理解了寄存器堆的内部结构、同步时序与多路控制逻辑。